

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
CHƯƠNG TRÌNH CAO HỌC



BÁO CÁO ĐỒ ÁN MÔN HỌC

Môn học: Hệ tính toán phân bố nâng cao — NT2204.CH201

TỐI ƯU HÓA THỜI GIAN KHỞI ĐỘNG
LẠNH CHO SUY DIỄN AI TRÊN NỀN
TẢNG SERVERLESS BẰNG CHIA SẺ
TRỌNG SỐ

Giảng viên hướng dẫn: TS. Huỳnh Văn Đặng

Học viên thực hiện: Ngô Quang Huy

TP. Hồ Chí Minh, Năm 2026

Tóm tắt đề án

Sự phát triển bùng nổ của deep learning đi kèm với thách thức lớn về độ trễ khởi động lạnh (cold-start) trong kiến trúc Serverless khi triển khai các mô hình có dung lượng lớn. Đề án này nghiên cứu và hiện thực hóa cơ chế chia sẻ trọng số (Weight-Sharing) thông qua việc cấu hình phân vùng `tmpfs` trên bộ nhớ RAM của node để làm lớp lưu trữ dùng chung cho các bản sao dịch vụ suy diễn (Pod). Sử dụng workload thực tế là mô hình nhận diện đối tượng YOLO-World kết hợp với framework API FastAPI trên hạ tầng Kubernetes (Minikube single-node) và framework Knative Serving.

Kết quả thực nghiệm giả lập bằng công cụ k6 và hệ thống giám sát Prometheus/Grafana cho thấy cơ chế tối ưu qua RAM giúp giảm trung bình 76.9% độ trễ P99 cold-start (từ 75.00 giây xuống còn 17.35 giây) so với việc đọc trực tiếp từ đĩa ảo cục bộ (Baseline). Đồng thời, hệ thống vận hành ổn định trong kịch bản tải bùng nổ (Burst Traffic) với tỷ lệ lỗi HTTP đạt mức 0.00%, đáp ứng toàn bộ các tiêu chí nghiệm thu đề ra, tạo tiền đề cải thiện hiệu năng xử lý thời gian thực cho hạ tầng Serverless AI.

Mục lục

Tóm tắt đồ án	i
Danh mục từ viết tắt	vi
1 Giới thiệu tổng quan	1
1.1 Mục tiêu đề tài	1
1.2 Tóm tắt nội dung thực hiện	2
2 Các công nghệ liên quan	4
2.1 Công nghệ 1: Kubernetes và Knative Serving	4
2.1.1 Khái niệm	4
2.1.2 Đặc điểm và Khả năng	4
2.1.3 Chức năng và Nhiệm vụ	4
2.1.4 Tại sao chọn nó?	5
2.2 Công nghệ 2: Mô hình YOLO-World và FastAPI	5
2.2.1 Khái niệm	5
2.2.2 Đặc điểm và Khả năng	5
2.2.3 Chức năng và Nhiệm vụ	5
2.2.4 Tại sao chọn nó?	6
2.3 Phương pháp: Cơ chế Weight-Sharing qua tmpfs (RAM)	6
2.3.1 Khái niệm	6
2.3.2 Đặc điểm và Khả năng	6
2.3.3 Chức năng và Nhiệm vụ	6
2.3.4 Tại sao chọn nó?	6
2.4 Công cụ kiểm thử và giám sát: k6, Prometheus và Grafana	7
2.4.1 Khái niệm	7
2.4.2 Đặc điểm và Khả năng	7
2.4.3 Chức năng và Nhiệm vụ	7
2.4.4 Tại sao chọn nó?	7
3 Hiện thực hoá	8
3.1 Kiến trúc tổng thể	8
3.1.1 Trình tạo tải k6	8
3.1.2 Cổng định tuyến Kourier	9
3.1.3 Bộ điều phối Knative Serving	9

3.1.4	Pod dịch vụ suy diễn	9
3.1.5	Lớp lưu trữ dùng chung	9
3.1.6	Hệ thống giám sát Prometheus và Grafana	10
3.2	Cài đặt và cấu hình cụ thể	10
3.2.1	Cấu hình phân vùng lưu trữ trên node	10
3.2.2	Cấu hình dịch vụ Knative	11
3.3	Phát triển ứng dụng và dịch vụ	12
3.3.1	Mã nguồn ứng dụng API cốt lõi	12
3.3.2	Đóng gói ứng dụng bằng Dockerfile	14
4	Các kịch bản thử nghiệm và kết quả	15
4.1	Kịch bản 1: Đánh giá thời gian khởi động lạnh (Cold-start)	15
4.1.1	Đặc tả	15
4.1.2	Kết quả	15
4.1.3	Thảo luận/đánh giá	16
4.2	Kịch bản 2: Khả năng chịu tải bùng nổ (Burst Traffic)	16
4.2.1	Đặc tả	16
4.2.2	Kết quả	17
4.2.3	Thảo luận/đánh giá	17
4.3	Kịch bản 3: Phân tích toàn chu kỳ (Cold - Burst - Warm)	18
4.3.1	Đặc tả	18
4.3.2	Kết quả	18
4.3.3	Thảo luận/đánh giá	19
4.4	Kịch bản 4: Giám sát nội bộ với Prometheus và Grafana	19
4.4.1	Đặc tả	19
4.4.2	Kết quả	19
4.4.3	Thảo luận/đánh giá	19
5	Kết luận và hướng phát triển	21
5.1	Kết luận	21
5.2	Hạn chế	21
5.3	Hướng phát triển	22
	Tài liệu tham khảo	23

Danh mục bảng

4.1	Thống kê thời gian phân vị P99 cold-start (Đơn vị: Giây)	16
4.2	Các chỉ số hiệu năng trong kịch bản Burst Traffic	17
4.3	Biến động độ trễ theo từng giai đoạn trong chu kỳ vận hành	18

Danh mục hình

3.1	Kiến trúc tổng thể hệ thống YOLO Serverless trên Minikube	8
3.2	Cơ chế Weight-Sharing qua hostPath: đối chứng giữa phân vùng tmpfs (RAM) và đĩa ảo trên node	10
4.1	So sánh độ trễ P99 cold-start giữa cơ chế Tối ưu (RAM) và Cơ sở (Disk)	16
4.2	Biểu đồ hiệu năng kịch bản Burst Traffic (độ trễ và trạng thái hệ thống theo thời gian)	17
4.3	Biến động độ trễ theo từng giai đoạn trong chu kỳ Cold – Burst – Warm	18
4.4	Dashboard Grafana giám sát metric yolo_model_load_seconds (cơ chế Tối ưu – RAM)	19

Danh mục từ viết tắt

AI	Artificial Intelligence (Trí tuệ nhân tạo)
API	Application Programming Interface (Giao diện lập trình ứng dụng)
ASGI	Asynchronous Server Gateway Interface
HTTP	Hypertext Transfer Protocol
I/O	Input/Output (Vào/Ra)
KPA	Knative Pod Autoscaler
MLOps	Machine Learning Operations
P99	Phân vị thứ 99 (99th Percentile)
SLA	Service Level Agreement (Cam kết chất lượng dịch vụ)
VU	Virtual User (Người dùng ảo)
YAML	YAML Ain't Markup Language

Chương 1

Giới thiệu tổng quan

Sự phát triển bùng nổ của AI, đặc biệt là các mô hình Deep Learning trong lĩnh vực Computer Vision, đã đặt ra những yêu cầu khắt khe về mặt hạ tầng triển khai. Cùng với đó, kiến trúc Serverless nổi lên như một giải pháp tối ưu hóa chi phí vận hành thông qua cơ chế cấp phát tài nguyên động và scale-to-zero khi hệ thống không có lưu lượng truy cập.

Tuy nhiên, việc kết hợp hai xu hướng công nghệ này vấp phải một rào cản kỹ thuật lớn: độ trễ cold-start. Đối với các mô hình AI có dung lượng weights lớn, mỗi khi hệ thống Serverless khởi tạo một container mới, quá trình nạp dữ liệu mô hình từ lưu trữ bền vững trên node vào bộ nhớ sẽ tiêu tốn một lượng thời gian đáng kể. Điều này làm gia tăng đột biến độ trễ, đặc biệt làm suy giảm nghiêm trọng chất lượng dịch vụ ở các phân vị trễ cao như P99, gây ảnh hưởng trực tiếp đến trải nghiệm người dùng trong các hệ thống yêu cầu xử lý thời gian thực.

Từ bối cảnh thực tiễn đó, đề án này được thực hiện nhằm đi sâu vào việc giải quyết nút thắt I/O dữ liệu, thông qua việc tái thiết kế cơ chế lưu trữ mô hình dùng chung trên cùng một node.

1.1 Mục tiêu đề tài

Mục tiêu cốt lõi của đề tài là nghiên cứu, thiết kế và triển khai một kiến trúc hạ tầng Serverless AI có khả năng giảm đáng kể độ trễ cold-start, tập trung vào tối ưu quá trình nạp weights mô hình.

Để hiện thực hóa mục tiêu cốt lõi trên, đề án xác định các mục tiêu cụ thể như sau:

- Xây dựng hạ tầng điều phối Serverless:** Thiết lập hệ thống Kubernetes trên môi trường giả lập (Minikube single-node) và tích hợp framework Knative Serving nhằm tạo ra một môi trường Serverless tiêu chuẩn, có khả năng autoscaling và scale-to-zero.
- Nghiên cứu và phát triển cơ chế Weight-Sharing:** Đề xuất giải pháp sử dụng hệ thống tệp tmpfs trên RAM làm vùng lưu trữ dùng chung giữa các Pod trên cùng một node. Mục tiêu là loại bỏ nút thắt I/O đĩa khi nạp mô hình trong giai đoạn container khởi động.
- Triển khai ứng dụng thử nghiệm thực tế:** Xây dựng một API suy diễn hình ảnh bằng framework FastAPI, sử dụng mô hình nhận diện đối tượng YOLO-World làm workload đại diện cho nhóm AI có dung lượng weights lớn.

4. **Đo lường, phân tích và đối chứng hiệu năng:** Thiết lập môi trường thử nghiệm so sánh trực tiếp giữa hai cơ chế đọc mô hình: từ RAM và từ đĩa ảo trên node. Sử dụng các công cụ chuyên dụng như k6 để giả lập các kịch bản lưu lượng, kết hợp cùng hệ sinh thái Prometheus và Grafana để thu thập, giám sát và phân tích định lượng.

1.2 Tóm tắt nội dung thực hiện

Để hoàn thành các mục tiêu đã đề ra, quá trình thực hiện đề án được chia thành các giai đoạn logic với những nội dung công việc cụ thể như sau:

- **Giai đoạn 1: Khởi tạo hạ tầng và đóng gói ứng dụng**
 - Khởi tạo cụm Kubernetes trên môi trường Minikube, cài đặt cấu hình định tuyến Kourier và Knative Serving.
 - Phát triển mã nguồn ứng dụng API bằng Python sử dụng FastAPI và thư viện Ultralytics YOLO.
 - Tích hợp mã đo lường tương thích với Prometheus ngay trong ứng dụng để theo dõi thời gian nạp mô hình.
 - Đóng gói toàn bộ mã nguồn và môi trường thực thi thành Docker image.
- **Giai đoạn 2: Hiện thực hóa cơ chế Weight-Sharing**
 - Xây dựng các bash script để cấu hình node, tiến hành cấp phát 4GB bộ nhớ RAM thành phân vùng tmpfs tại đường dẫn /mnt/shared-weights.
 - Tải trực tiếp tệp trọng số yolov8l-world.pt vào cả phân vùng RAM và phân vùng đĩa ảo trên node (/mnt/disk-weights) để làm cơ sở đối chứng.
 - Khai báo cấu hình YAML cho hai dịch vụ Knative riêng biệt, ánh xạ đúng đường dẫn thư mục tương ứng theo cơ chế hostPath.
- **Giai đoạn 3: Giả lập lưu lượng và đo lường**
 - Viết các kịch bản load test bằng k6 để giả lập ba trạng thái tải đặc thù: cold-start độc lập, burst traffic, và chu kỳ đầy đủ cold - burst - warm.
 - Triển khai hệ thống Prometheus và Grafana để giám sát. Xây dựng dashboard trực quan đối chiếu P99 và thời gian nạp model giữa RAM và đĩa.
- **Giai đoạn 4: Tổng hợp và đánh giá số liệu**
 - Thực thi kịch bản cold-start 3 lần độc lập để đảm bảo độ tin cậy của phân phối thống kê.

- Phân tích số liệu từ k6 và Grafana, rút ra kết luận cốt lõi: Việc áp dụng Weight-Sharing qua RAM đã giúp giảm khoảng 76.9% độ trễ P99 cold-start trung bình (từ 75.00s xuống 17.35s). Đồng thời, trong kịch bản burst traffic, hệ thống xử lý ổn định tải bùng nổ mà không phát sinh lỗi kết nối HTTP (tỷ lệ lỗi 0%) và không bị tắc nghẽn theo tiêu chí nghiệm thu.

Chương 2

Các công nghệ liên quan

Chương này trình bày các cơ sở lý thuyết và đặc tính kỹ thuật của các công nghệ cốt lõi được áp dụng trong đề án. Nội dung tập trung vào hệ sinh thái điều phối vùng chứa, các thư viện phục vụ AI và phương pháp tối ưu hóa bộ nhớ nhằm giải quyết bài toán cold-start.

2.1 Công nghệ 1: Kubernetes và Knative Serving

2.1.1 Khái niệm

Kubernetes là một nền tảng mã nguồn mở dùng để tự động hóa việc triển khai, mở rộng và quản lý các ứng dụng dưới dạng vùng chứa (container). Trong hệ sinh thái này, **Knative Serving** đóng vai trò là một lớp trừu tượng hóa mức cao, cung cấp các tính năng Serverless cho Kubernetes, cho phép các nhà phát triển tập trung vào mã nguồn ứng dụng thay vì phải tự cấu hình hạ tầng mạng và tài nguyên phức tạp.

2.1.2 Đặc điểm và Khả năng

Knative Serving kết hợp hệ sinh thái của Kubernetes với mô hình Serverless để cung cấp các cơ chế:

- **Thu hồi tài nguyên triệt để (Scale-to-zero):** Tự động giảm số lượng Pod về 0 khi hệ thống không nhận được yêu cầu nào trong một khoảng thời gian được cấu hình trước, giúp tối ưu hóa tài nguyên tính toán.
- **Mở rộng theo hướng yêu cầu (Request-driven Autoscaling):** Khác với cơ chế mở rộng theo CPU/RAM, Knative có khả năng khởi tạo hoặc nhân bản Pod dựa trên số lượng yêu cầu HTTP (concurrency) thực tế đang được gửi đến hệ thống.
- **Quản lý phiên bản (Revision Tracking):** Mỗi lần cập nhật dịch vụ, hệ thống tạo ra một phiên bản bất biến, hỗ trợ việc cập nhật và điều phối lưu lượng mạng.

2.1.3 Chức năng và Nhiệm vụ

Trong phạm vi đề án, Kubernetes đóng vai trò là hạ tầng điều phối nền tảng trên môi trường Minikube, quản lý vòng đời của các Pod và cung cấp cơ chế hostPath để gắn kết các phân

vùng bộ nhớ cục bộ. Knative Serving chịu trách nhiệm thực thi cơ chế Serverless, kiểm soát quá trình khởi động Pod từ trạng thái nghỉ khi hệ thống bắt đầu nhận lưu lượng và tự động điều phối autoscaling để hấp thụ tải bùng nổ.

2.1.4 Tại sao chọn nó?

Kubernetes hiện là một tiêu chuẩn phổ biến về điều phối container. Knative Serving được lựa chọn thay vì các giải pháp đám mây đóng kín (như AWS Lambda) vì đây là nền tảng mã nguồn mở, cho phép người quản trị can thiệp vào tầng cấu hình hạ tầng. Sự can thiệp này là cần thiết để đồ án có thể cấp phát bộ nhớ RAM cục bộ và cấu hình đường dẫn đọc mô hình cho cơ chế Weight-Sharing.

2.2 Công nghệ 2: Mô hình YOLO-World và FastAPI

2.2.1 Khái niệm

Ultralytics YOLO-World là một biến thể trong họ mô hình nhận diện đối tượng YOLO, được thiết kế để phát hiện đối tượng dựa trên mô tả ngôn ngữ tự nhiên (tập từ vựng mở - open-vocabulary). **FastAPI** là một framework web dành cho Python, được xây dựng dựa trên các tiêu chuẩn ASGI, hỗ trợ xử lý bất đồng bộ mạnh mẽ, giảm thiểu tối đa overhead của tầng giao tiếp mạng.

2.2.2 Đặc điểm và Khả năng

Đặc tính kỹ thuật quan trọng của YOLO-World trong ngữ cảnh đồ án này là việc sử dụng các tệp trọng số (weights) có kích thước lớn (khoảng 90MB cho phiên bản Large), tạo ra áp lực I/O đáng kể khi container khởi động. Về phía API, FastAPI có tốc độ thực thi cao, kiến trúc bất đồng bộ và hỗ trợ tích hợp các thư viện đo lường như Prometheus client.

2.2.3 Chức năng và Nhiệm vụ

Mô hình YOLO-World đóng vai trò là tải trọng công việc (workload) thực tế của hệ thống, là đối tượng chính cần được đo lường thời gian nạp trong giai đoạn cold-start. FastAPI được sử dụng để xây dựng lớp giao diện web, làm nhiệm vụ tiếp nhận hình ảnh, gọi YOLO-World để suy diễn và trả về kết quả JSON, đồng thời duy trì một endpoint riêng để hệ thống giám sát có thể thu thập (scrape) số liệu runtime.

2.2.4 Tại sao chọn nó?

YOLO-World được chọn vì nó là đại diện phù hợp cho các mô hình AI có khối lượng tệp trọng số tương đối lớn, giúp bộc lộ giới hạn của việc đọc dữ liệu từ đĩa ảo trên node. FastAPI được sử dụng nhằm thu gọn lớp overhead của framework web, đảm bảo hiệu năng xử lý HTTP không trở thành nút thắt của hệ thống.

2.3 Phương pháp: Cơ chế Weight-Sharing qua tmpfs (RAM)

2.3.1 Khái niệm

tmpfs (Temporary File System) là một hệ thống tệp tạm thời trong hệ điều hành Linux, được lưu trữ trên bộ nhớ truy cập ngẫu nhiên (RAM). Phương pháp Weight-Sharing qua tmpfs sử dụng vùng nhớ này để nạp sẵn tệp mô hình AI và cấu hình để các container chia sẻ quyền truy cập đọc.

2.3.2 Đặc điểm và Khả năng

Hệ thống tệp tmpfs cung cấp băng thông cao và độ trễ thấp hơn rõ rệt so với tốc độ đọc tuần tự của lưu trữ bền vững (đĩa ảo trên node). Dữ liệu trên tmpfs mang tính bay hơi, nhưng trong kiến trúc single-node, nó cho phép nhiều Pod dùng chung một bản sao mô hình duy nhất trên RAM, qua đó hệ thống không đọc từ đĩa ảo trên node.

2.3.3 Chức năng và Nhiệm vụ

Đây là cơ chế kỹ thuật trọng tâm của đồ án để giảm độ trễ cold-start. Nhiệm vụ của cơ chế này là cung cấp một đường dẫn đọc weights tốc độ cao (/mnt/shared-weights). Thay vì đọc từ đĩa ảo trên node, container đọc trực tiếp từ tmpfs trên RAM, giúp giảm đáng kể chi phí I/O khi khởi tạo mô hình — dù chi phí giải nén và nạp vào không gian nhớ của runtime (deserialize) vẫn còn tồn tại.

2.3.4 Tại sao chọn nó?

Cơ chế này được chọn vì nó giải quyết trực tiếp một thành phần gây ra hiện tượng cold-start chậm chạp của AI Serverless: quá trình đọc tệp từ đĩa ảo trên node. Trong môi trường single-node, việc sử dụng tmpfs là phương pháp trực quan để thiết lập môi trường đối chứng giữa RAM và đĩa, đồng thời mang tính minh họa cho nguyên lý phân lớp lưu trữ (Tiered Storage) ở mức cục bộ.

2.4 Công cụ kiểm thử và giám sát: k6, Prometheus và Grafana

2.4.1 Khái niệm

k6 là một công cụ mã nguồn mở dùng để kiểm thử tải hệ thống bằng kịch bản JavaScript. **Prometheus** là cơ sở dữ liệu chuỗi thời gian (time-series) hỗ trợ thu thập và lưu trữ metrics, trong khi **Grafana** là nền tảng trực quan hóa dữ liệu hiển thị.

2.4.2 Đặc điểm và Khả năng

k6 hỗ trợ giả lập lượng người dùng ảo (VU) và định nghĩa các kịch bản tải phức tạp. Prometheus hoạt động theo cơ chế kéo (pull model) thông qua việc gọi các endpoint `/metrics`. Grafana kết nối trực tiếp với Prometheus để tạo các biểu đồ (Dashboard) theo thời gian thực.

2.4.3 Chức năng và Nhiệm vụ

Các công cụ được phân chia nhiệm vụ rõ ràng: k6 đóng vai trò là client gửi lưu lượng từ bên ngoài, ghi nhận độ trễ phản hồi tổng thể (end-to-end latency) của toàn bộ quá trình (bao gồm cold-start, thời gian suy diễn và độ trễ mạng). Ở phía hạ tầng nội bộ, Prometheus chịu trách nhiệm thu thập (scrape) số liệu thời gian nạp model cục bộ (metric `yolo_model_load_seconds`) từ FastAPI, sau đó Grafana xuất ra các biểu đồ đối chiếu trực quan.

2.4.4 Tại sao chọn nó?

Bộ công cụ này tạo thành một pipeline đo lường minh bạch. k6 cho phép giả lập các pha tải từ cold-start đến burst traffic một cách linh hoạt. Sự kết hợp giữa góc nhìn bên ngoài của k6 và khả năng giám sát nội tại của Prometheus mang lại tính định lượng cao, đảm bảo các kết quả đo lường có thể được kiểm chứng và tái lập độc lập.

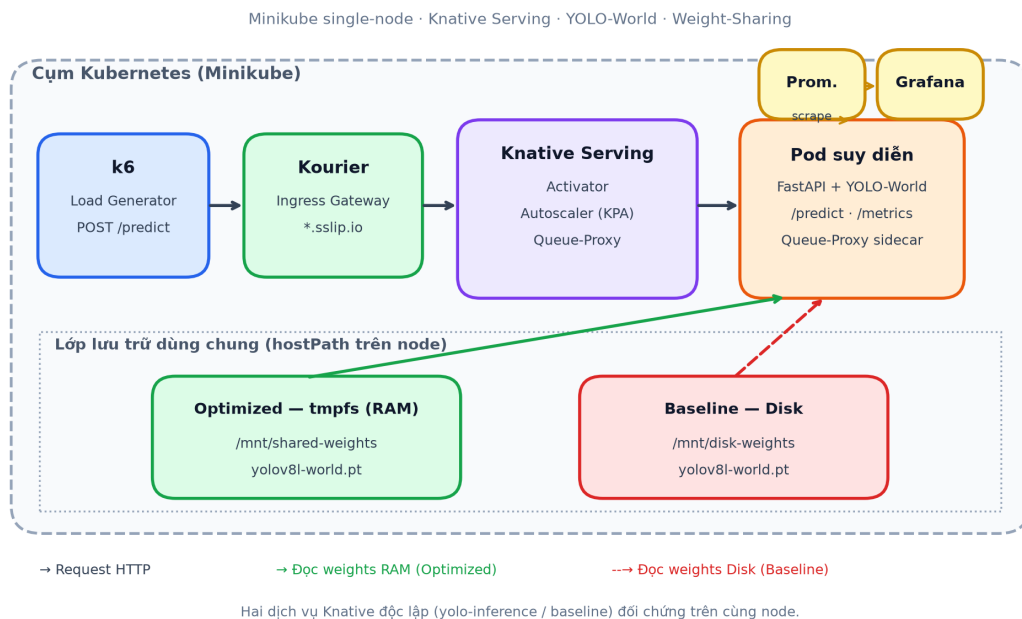
Chương 3

Hiện thực hoá

Chương này trình bày chi tiết quá trình hiện thực hóa hệ thống, bao gồm thiết kế kiến trúc tổng thể, các bước cài đặt, cấu hình hạ tầng và quy trình phát triển ứng dụng suy diễn. Các cấu hình kỹ thuật và phân đoạn mã nguồn quan trọng được trích xuất từ kho lưu trữ (repository) nhằm làm rõ cơ chế chia sẻ trọng số và phương pháp thu thập dữ liệu hiệu năng.

3.1 Kiến trúc tổng thể

Hệ thống nhận diện đối tượng Serverless được thiết kế theo mô hình kiến trúc phân lớp hướng dịch vụ. Toàn bộ các thành phần được triển khai tập trung trên một node của cụm Minikube (môi trường máy ảo) để đảm bảo tính cục bộ của dữ liệu. Sơ đồ kiến trúc tổng thể của hệ thống được minh họa ở Hình 3.1, bao gồm sáu thành phần chính sau:



Hình 3.1: Kiến trúc tổng thể hệ thống YOLO Serverless trên Minikube

3.1.1 Trình tạo tải k6

Đây là tác nhân kiểm thử nằm ở biên ngoài của hệ thống. k6 thực thi các kịch bản load test bằng cách gửi liên tục các yêu cầu HTTP POST chứa dữ liệu hình ảnh nhị phân đến endpoint

/predict. k6 chịu trách nhiệm đo lường độ trễ end-to-end từ góc nhìn của người dùng, ghi nhận tỷ lệ lỗi kết nối và thống kê các phân vị độ trễ phục vụ cho việc đánh giá hiệu năng.

3.1.2 Cổng định tuyến Kourier

Kourier đóng vai trò là Ingress Gateway cho toàn bộ cụm Knative. Thông qua việc sử dụng `kubectl port-forward`, Kourier tiếp nhận các yêu cầu HTTP từ k6 bên ngoài cụm, định tuyến lưu lượng dựa trên tên miền ảo (như `*.sslip.io`) và chuyển tiếp đến các dịch vụ Knative tương ứng.

3.1.3 Bộ điều phối Knative Serving

Đây là hệ thống điều khiển cơ chế Serverless, quản lý luồng autoscaling thông qua ba thành phần chính:

- **Autoscaler (KPA - Knative Pod Autoscaler):** Thu thập các chỉ số về concurrency và tính toán số lượng Pod cần thiết (replica) để đáp ứng mức tải hiện tại.
- **Activator:** Đóng vai trò là điểm hứng yêu cầu tạm thời khi số lượng Pod đang ở mức 0. Nó sẽ giữ các yêu cầu đầu tiên của k6 lại, kích hoạt quá trình scale-from-zero, và chỉ chuyển tiếp yêu cầu đi khi Pod đã chuyển sang trạng thái sẵn sàng (Ready).
- **Queue-Proxy:** Một container phụ chạy song hành (sidecar) bên trong mỗi Pod để giám sát lưu lượng thực tế và giới hạn số lượng yêu cầu đồng thời đi vào container chính.

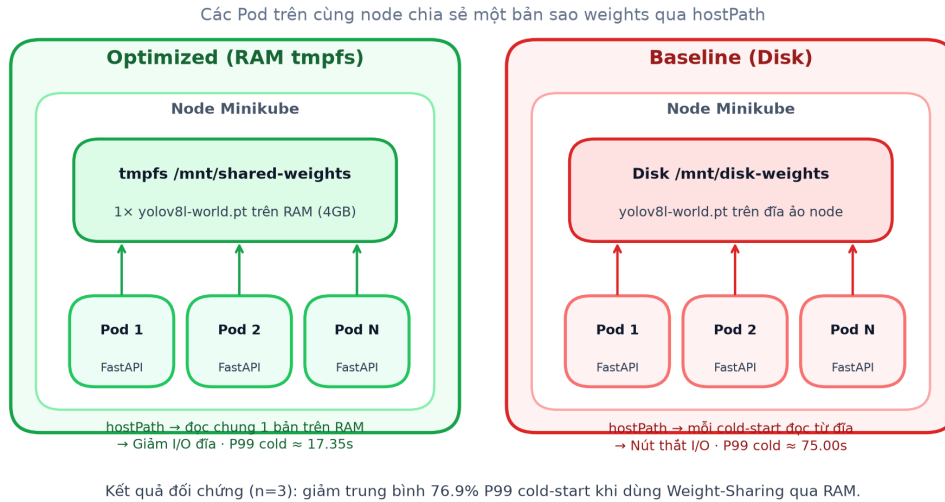
3.1.4 Pod dịch vụ suy diễn

Là đơn vị tính toán nhỏ nhất được quản lý bởi Kubernetes. Mỗi Pod chứa container chính chạy ứng dụng FastAPI xử lý logic suy diễn YOLO-World và container phụ Queue-Proxy của Knative.

3.1.5 Lớp lưu trữ dùng chung

Thành phần cốt lõi quyết định tốc độ nạp weights của mô hình. Hệ thống sử dụng cơ chế `hostPath` của Kubernetes để liên kết trực tiếp thư mục từ node của cụm vào bên trong các Pod. Có hai phân vùng lưu trữ được thiết lập song song để phục vụ quá trình đối chứng:

- Phân vùng `tmpfs` được cấp phát trên RAM của node tại đường dẫn `/mnt/shared-weights`, đại diện cho cơ chế tối ưu hóa (Optimized).
- Thư mục lưu trữ bền vững trên đĩa node tại đường dẫn `/mnt/disk-weights`, đại diện cho cơ chế cơ sở (Baseline).



Hình 3.2: Cơ chế Weight-Sharing qua hostPath: đối chứng giữa phân vùng tmpfs (RAM) và đĩa ảo trên node

3.1.6 Hệ thống giám sát Prometheus và Grafana

Prometheus thực hiện cào (scrape) định kỳ endpoint `/metrics` của các Pod đang hoạt động để thu thập các chỉ số hiệu năng nội bộ (thời gian nạp model và biểu đồ phân phối độ trễ API). Grafana kết nối với Prometheus để trực quan hóa các biểu đồ đối sánh hiệu năng trực tiếp giữa hai cơ chế RAM và đĩa.

3.2 Cài đặt và cấu hình cụ thể

Phần lớn quy trình cài đặt weights, build image và apply YAML được tự động hóa bằng các shell script (`scripts/setup-weights.sh`, `scripts/deploy-all.sh`) nhằm đảm bảo khả năng tái lập kết quả thử nghiệm.

3.2.1 Cấu hình phân vùng lưu trữ trên node

Hệ thống can thiệp vào node Minikube thông qua SSH. Trích đoạn logic từ `scripts/setup-weights.sh` cho thấy các bước cấp phát RAM và tạo thư mục baseline trên đĩa node:

```

1 # Check and allocate 4GB tmpfs partition if not already mounted
2 if ! minikube ssh "mountpoint -q /mnt/shared-weights"; then
3     minikube ssh "sudo mount -t tmpfs -o size=4G tmpfs /mnt/shared-weights"
4 fi
5

```

```

6 # Download the yolov8l-worldv2.pt weight file and save as yolov8l-
  world.pt
7 # and set up a parallel copy in the /mnt/disk-weights directory
  for baseline comparison

```

Listing 3.1: Trích đoạn cấu hình cấp phát tài nguyên RAM tmpfs trên Node

3.2.2 Cấu hình dịch vụ Knative

Hệ thống định nghĩa hai dịch vụ Knative độc lập. Dưới đây là trích đoạn cấu hình cốt lõi từ `service.yaml` (dịch vụ tối ưu):

```

1 apiVersion: serving.knative.dev/v1
2 kind: Service
3 metadata:
4   name: yolo-inference
5   labels:
6     deployment-mode: optimized
7 spec:
8   template:
9     metadata:
10      annotations:
11        autoscaling.knative.dev/min-scale: "0"
12        autoscaling.knative.dev/max-scale: "5"
13        autoscaling.knative.dev/target: "1"
14        prometheus.io/scrape: "true"
15        prometheus.io/port: "8080"
16        prometheus.io/path: "/metrics"
17     spec:
18       containers:
19         - image: dev.local/yolo-serverless-api:v4
20           imagePullPolicy: Never
21           ports:
22             - containerPort: 8080
23           env:
24             - name: MODEL_PATH
25               value: "/mnt/shared-weights/yolov8l-world.pt"
26             - name: DEPLOYMENT_MODE
27               value: "optimized"
28           volumeMounts:
29             - name: model-volume

```

```

30         mountPath: /mnt/shared-weights
31     volumes:
32     - name: model-volume
33       hostPath:
34         path: /mnt/shared-weights
35         type: Directory

```

Listing 3.2: Cấu hình YAML triển khai Knative Serving cho dịch vụ Tối ưu hóa

Đối với dịch vụ cơ sở (`service-baseline.yaml`), các giá trị được điều chỉnh thành `baseline` và trở đường dẫn `hostPath` về thư mục đĩa `/mnt/disk-weights`. Cấu hình `imagePullPolicy`: `Never` được áp dụng khắt khe để ép hệ thống sử dụng ảnh cục bộ, loại trừ biến số độ trễ tải mạng trong quá trình đo lường `cold-start`.

3.3 Phát triển ứng dụng và dịch vụ

Ứng dụng phục vụ suy diễn được phát triển bằng ngôn ngữ Python sử dụng framework `FastAPI` nhằm tối ưu hóa tốc độ xử lý HTTP.

3.3.1 Mã nguồn ứng dụng API cốt lõi

Điểm đặc biệt trong kiến trúc của ứng dụng là việc **nạp mô hình ngay tại mức top-level (module import)**. Cách tiếp cận này khiến chi phí thời gian nạp mô hình xảy ra ngay khi process Python khởi động, nghĩa là toàn bộ I/O nạp file sẽ diễn ra và kết thúc *trước* khi Pod báo cáo trạng thái Ready cho Kubernetes.

Dưới đây là trích đoạn mã nguồn thực tế tập trung vào phần nạp mô hình và đo lường từ `app.py`:

```

1  import os
2  import time
3  import json
4  import io
5  from PIL import Image
6  from fastapi import FastAPI, File, UploadFile, Response
7  from fastapi.responses import JSONResponse
8  from ultralytics import YOLOWorld
9  from prometheus_client import generate_latest, CONTENT_TYPE_LATEST
10     , Gauge, Counter, Histogram
11  # Configure environment variables

```

```

12 MODEL_PATH = os.environ.get("MODEL_PATH", "/mnt/shared-weights/
    yolo_v8l-world.pt")
13 DEPLOYMENT_MODE = os.environ.get("DEPLOYMENT_MODE", "optimized")
14
15 app = FastAPI()
16
17 # Declare Prometheus Metrics
18 MODEL_LOAD_SECONDS = Gauge('yolo_model_load_seconds', 'Model
    loading time', ['mode'])
19 REQUEST_LATENCY = Histogram('
    yolo_inference_request_duration_seconds', 'Inference duration',
    ['mode'])
20 REQUEST_COUNT = Counter('yolo_inference_requests_total', 'Total
    requests', ['mode', 'status'])
21
22 # LOAD MODEL INTO MEMORY AT MODULE STARTUP (Top-level)
23 start_time = time.time()
24 model = YOLOWorld(MODEL_PATH)
25 load_duration = time.time() - start_time
26 MODEL_LOAD_SECONDS.labels(mode=DEPLOYMENT_MODE).set(load_duration)
27
28 @app.post("/predict")
29 async def predict(file: UploadFile = File(...)):
30     start_req = time.time()
31     status = "success"
32     try:
33         image_bytes = await file.read()
34         image = Image.open(io.BytesIO(image_bytes))
35
36         results = model.predict(image)
37         detections = json.loads(results[0].to_json())
38
39         return {
40             "status": status,
41             "mode": DEPLOYMENT_MODE,
42             "detections": detections
43         }
44     except Exception:
45         status = "error"
46         raise

```

```

47     finally:
48         req_duration = time.time() - start_req
49         REQUEST_LATENCY.labels(mode=DEPLOYMENT_MODE).observe(
50             req_duration)
51         REQUEST_COUNT.labels(mode=DEPLOYMENT_MODE, status=status).
52             inc()
53
54 @app.get("/health")
55 def health_check():
56     return {"status": "ok", "mode": DEPLOYMENT_MODE, "model_path":
57         MODEL_PATH}
58
59 @app.get("/metrics")
60 def metrics():
61     return Response(generate_latest(), media_type=
62         CONTENT_TYPE_LATEST)

```

Listing 3.3: Mã nguồn ứng dụng app.py xử lý nạp model và tích hợp Prometheus metrics

3.3.2 Đóng gói ứng dụng bằng Dockerfile

Môi trường ứng dụng được đóng gói tinh gọn dựa trên nền tảng python:3.10-slim. Trích đoạn Dockerfile mô tả quá trình cài đặt hệ thống và thư viện:

```

1 FROM python:3.10-slim
2
3 # Install system dependencies for OpenCV
4 RUN apt-get update && apt-get install -y libgl1 libglib2.0-0 && rm
5     -rf /var/lib/apt/lists/*
6
7 WORKDIR /app
8 COPY requirements.txt .
9 RUN pip install --no-cache-dir -r requirements.txt
10
11 COPY app.py .
12
13 EXPOSE 8080
14 CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8080"]

```

Listing 3.4: Dockerfile đóng gói môi trường API runtime

Chương 4

Các kịch bản thử nghiệm và kết quả

Chương này trình bày chi tiết các phương pháp đo lường, kết quả thực nghiệm và các phân tích định lượng nhằm đánh giá hiệu năng của hệ thống Serverless AI. Các thử nghiệm được thực thi trên môi trường Minikube. Chỉ số quan trọng nhất được sử dụng để so sánh là độ trễ ở phân vị thứ 99 (P99 Latency), phản ánh thời gian chờ đợi của nhóm 1% các yêu cầu có thời gian phản hồi chậm nhất.

4.1 Kịch bản 1: Đánh giá thời gian khởi động lạnh (Cold-start)

4.1.1 Đặc tả

Kịch bản này nhằm định lượng ảnh hưởng của quá trình nạp trọng số mô hình đến độ trễ cold-start.

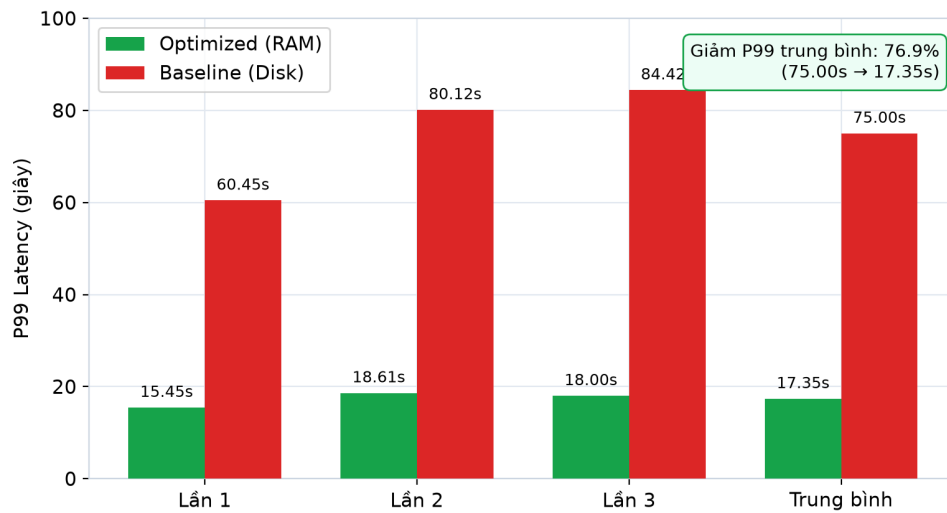
- **Công cụ và thông số:** Sử dụng script `loadtest-cold.js` của k6. Trong pipeline nghiệm thu chính thức, số lượng người dùng ảo được thiết lập ở mức `COLD_VUS=3` (3 yêu cầu đồng thời).
- **Điều kiện tiên quyết:** Trước mỗi lần chạy, hệ thống được ép buộc thu hồi tài nguyên (`scale-to-zero`).
- **Phương pháp đối chứng:** Thử nghiệm được lặp lại 3 lần độc lập ($n=3$) cho mỗi cơ chế lưu trữ: Tối ưu hóa (Optimized - đọc từ RAM `tmpfs`) và Cơ sở (Baseline - đọc từ đĩa ảo trên node). Cỡ mẫu hạn chế ở $n=3$ là sự đánh đổi có chủ ý nhằm tối ưu tài nguyên phần cứng trong môi trường giả lập cục bộ.

4.1.2 Kết quả

Dữ liệu thu thập từ 3 lần chạy độc lập cho thấy sự chênh lệch rõ rệt về thời gian khởi động hệ thống giữa hai cơ chế được thể hiện ở Bảng 4.1:

Bảng 4.1: Thống kê thời gian phân vị P99 cold-start (Đơn vị: Giây)

Lượt chạy	Cơ chế Tối ưu (RAM)	Cơ chế Cơ sở (Disk)
Lần 1	15.45	60.45
Lần 2	18.61	80.12
Lần 3	18.00	84.42
Trung bình	17.35	75.00
Độ lệch chuẩn	1.68	12.78



Hình 4.1: So sánh độ trễ P99 cold-start giữa cơ chế Tối ưu (RAM) và Cơ sở (Disk)

4.1.3 Thảo luận/đánh giá

Việc áp dụng cơ chế Weight-Sharing qua RAM đã giúp giảm trung bình 76.9% độ trễ P99 cold-start so với đọc từ đĩa ảo trên node (từ 75.00s xuống còn 17.35s). Với mẫu $n=3$, giá trị P99 thu được gần với giá trị cực đại, mang tính minh họa cho sự ổn định tương đối của cơ chế Optimized so với Baseline. Kết quả này xác nhận rằng việc giảm chi phí I/O đọc tệp trọng số là yếu tố then chốt để cải thiện hiệu năng khởi động. Tuy nhiên, thời gian 17.35s trên RAM cho thấy chi phí lập lịch Pod, khởi tạo runtime Python và giải nén dữ liệu (deserialize) vẫn là các thành phần hao tổn tài nguyên đáng kể của kiến trúc Serverless hiện tại.

4.2 Kịch bản 2: Khả năng chịu tải bùng nổ (Burst Traffic)

4.2.1 Đặc tả

Kịch bản này đánh giá khả năng duy trì dịch vụ khi đối mặt với lượng tải gia tăng đột biến.

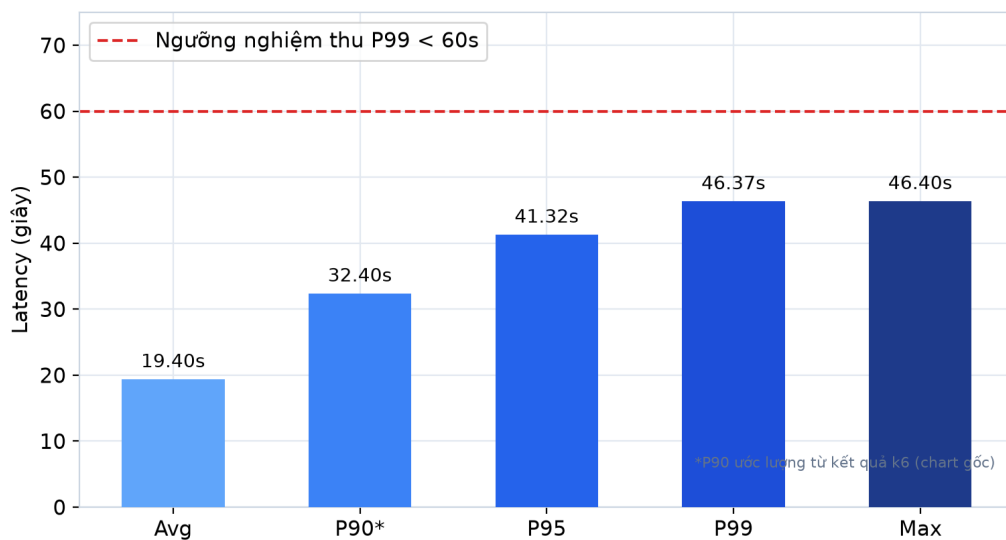
- **Công cụ và thông số:** Sử dụng script `loadtest-burst.js` với hồ sơ tải 3 pha: tăng dần (ramp-up) trong 15s lên 15 VUs, giữ tải 60s, và giảm dần (ramp-down) trong 15s.
- **Tiêu chí nghiệm thu:** Tỷ lệ lỗi HTTP < 10% và P99 Latency < 60s.

4.2.2 Kết quả

Hệ thống xử lý tổng cộng 61 yêu cầu suy diễn với các thông số đo lường tại Bảng 4.2:

Bảng 4.2: Các chỉ số hiệu năng trong kịch bản Burst Traffic

Chỉ số đo lường	Giá trị thu nhận
Tỷ lệ lỗi kết nối (HTTP Error Rate)	0.00%
Thời gian phản hồi trung bình (Avg)	19.40 s
Độ trễ phân vị 99 (P99)	46.37 s
Trạng thái hệ thống	Không tắc nghẽn (Congested: false)



Hình 4.2: Biểu đồ hiệu năng kịch bản Burst Traffic (độ trễ và trạng thái hệ thống theo thời gian)

4.2.3 Thảo luận/đánh giá

Hệ thống đáp ứng tốt các tiêu chí nghiệm thu. Cơ chế autoscaling của Knative đã điều tiết hiệu quả lưu lượng bùng nổ trong giới hạn `max-scale: 5`, không từ chối yêu cầu. Mức P99 46.37s nằm trong ngưỡng cho phép (dưới 60s), phản ánh năng lực xử lý suy diễn của mô hình trên tài nguyên tính toán cục bộ.

4.3 Kịch bản 3: Phân tích toàn chu kỳ (Cold - Burst - Warm)

4.3.1 Đặc tả

Mô phỏng vòng đời toàn diện của dịch vụ:

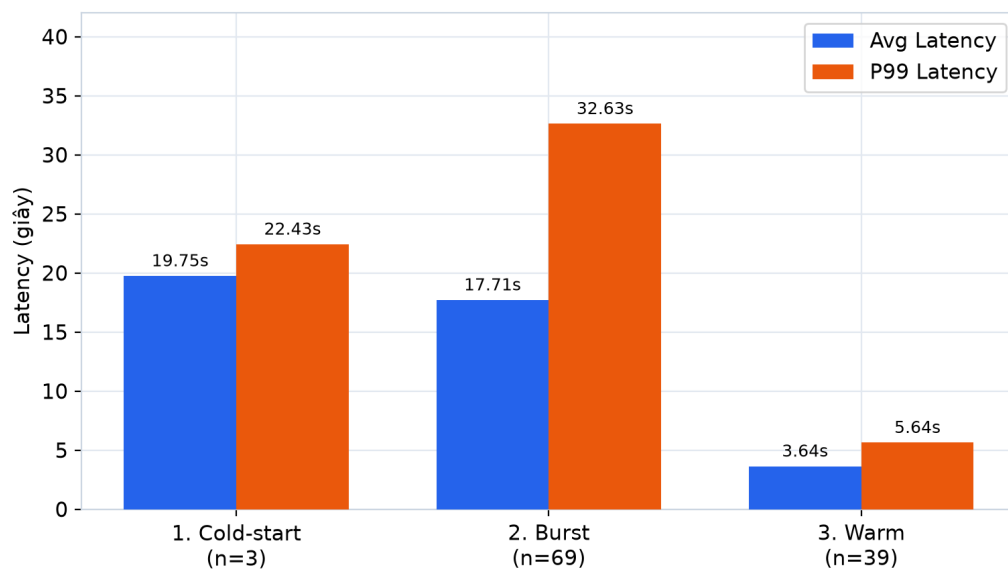
- Pha Cold-start: 3 VUs gửi yêu cầu đồng thời.
- Pha Burst: Tăng dần lên 15 VUs sau thời gian chờ.
- Pha Warm: Duy trì 5 VUs gửi yêu cầu đều đặn trong 1 phút khi Pod đã sẵn sàng.

4.3.2 Kết quả

Tổng cộng thu được 111 yêu cầu thành công (tỷ lệ lỗi 0%) được chia cụ thể theo từng giai đoạn ở Bảng 4.3:

Bảng 4.3: Biến động độ trễ theo từng giai đoạn trong chu kỳ vận hành

Giai đoạn	Số yêu cầu	Trễ trung bình (Avg)	Phân vị P99
1. Khởi động (Cold)	3	19.75 s	22.43 s
2. Bùng nổ (Burst)	69	17.71 s	32.63 s
3. Ổn định (Warm)	39	3.64 s	5.64 s



Hình 4.3: Biến động độ trễ theo từng giai đoạn trong chu kỳ Cold – Burst – Warm

4.3.3 Thảo luận/đánh giá

Khi ở pha Warm, độ trễ P99 đạt 5.64s. Khoảng cách chênh lệch giữa pha Cold (22.43s) and pha Warm (5.64s) giúp ước lượng tổng chi phí hạ tầng cho một lần khởi tạo Pod mới trên hệ thống. Dữ liệu này là cơ sở thực tế để người quản trị thiết lập ngưỡng SLA cho người dùng cuối.

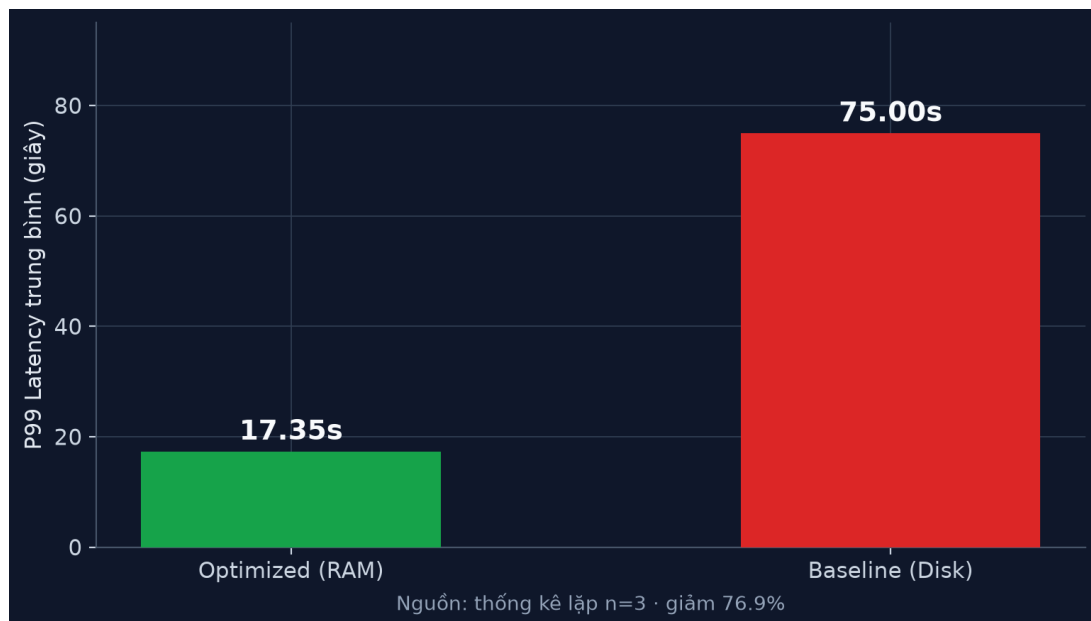
4.4 Kịch bản 4: Giám sát nội bộ với Prometheus và Grafana

4.4.1 Đặc tả

Đánh giá hiệu năng nội sinh bên trong container thông qua metric `yolo_model_load_seconds` thu thập bởi Prometheus.

4.4.2 Kết quả

Đối với cơ chế Tối ưu (RAM tmpfs): Thời gian nạp tệp ổn định ở mức **1.38 giây**, được trực quan hóa qua dashboard Grafana như Hình 4.4.



Hình 4.4: Dashboard Grafana giám sát metric `yolo_model_load_seconds` (cơ chế Tối ưu – RAM)

4.4.3 Thảo luận/đánh giá

Mức 1.38s nạp tệp 90MB từ RAM khẳng định hiệu quả bằng thông của `tmpfs`. Khi so sánh con số này với tổng độ trễ cold-start 17.35s (ở kịch bản 1), có thể thấy tác vụ nạp mô hình thuần túy chỉ chiếm chưa tới 10% chi phí khởi động trong cấu hình tối ưu. Điều này cho thấy sau khi

loại bỏ nút thắt I/O đĩa, các giới hạn tiếp theo của hệ thống nằm ở quá trình lập lịch và khởi tạo runtime của hạ tầng container.

Chương 5

Kết luận và hướng phát triển

5.1 Kết luận

Đồ án đã thực hiện nghiên cứu, thiết kế và triển khai thành công một kiến trúc hạ tầng AI Serverless trên nền tảng Kubernetes và Knative, tập trung giải quyết bài toán độ trễ khởi động lạnh (cold-start) trong các mô hình nhận diện đối tượng có dung lượng trọng số lớn. Những kết quả đạt được bao gồm:

- **Tối ưu hóa hiệu năng khởi động:** Bằng việc đề xuất và hiện thực hóa cơ chế Weight-Sharing qua hệ thống tập tmpfs trên RAM, đồ án đã đạt được kết quả ấn tượng: giảm trung bình 76.9% độ trễ P99 cold-start (từ 75.00 giây xuống còn 17.35 giây) so với phương pháp đọc từ đĩa ảo trên node. Với mẫu thử n=3, giá trị P99 phản ánh rõ rệt mức cải thiện và độ ổn định tương đối giữa hai cơ chế.
- **Đảm bảo tính ổn định của hệ thống:** Thông qua việc kiểm thử tải bùng nổ (burst traffic), hệ thống chứng minh khả năng vận hành ổn định trong giới hạn cấu hình (max-scale: 5), không phát sinh lỗi kết nối HTTP (tỷ lệ lỗi 0%) và không rơi vào trạng thái tắc nghẽn (P99 46.37s < 60s), đáp ứng tốt các tiêu chí nghiệm thu đề ra.
- **Thiết lập pipeline giám sát tiêu chuẩn:** Đồ án đã xây dựng thành công hệ sinh thái giám sát MLOps sử dụng k6, Prometheus và Grafana. Hệ thống này cho phép quan sát đồng thời độ trễ từ góc nhìn *end-user* và các chỉ số nội tại (như `yolo_model_load_seconds`), tạo tiền đề cho việc quản lý và tối ưu hóa hệ thống.

Tóm lại, đồ án đã giảm đáng kể nút thắt I/O đĩa khi nạp trọng số AI trên hạ tầng Serverless trong môi trường thực nghiệm. Kết quả thực nghiệm khẳng định việc đặt thành phần "nóng" (tập trọng số) trên RAM là chiến lược hiệu quả, dù chi phí lập lịch Pod và khởi tạo runtime vẫn còn chiếm phần lớn thời gian khởi động sau khi đã tối ưu I/O.

5.2 Hạn chế

Bên cạnh những kết quả đạt được, đồ án vẫn còn tồn tại một số hạn chế cần được xem xét:

- **Phạm vi triển khai:** Hệ thống hiện tại được tối ưu trên Minikube single-node, mô phỏng nguyên lý phân lớp lưu trữ cục bộ, chưa triển khai trên cụm máy chủ phân tán đa node.

- **Giới hạn bộ nhớ:** Phân vùng `tmpfs` 4GB hiện là cấu hình tĩnh; việc cấp phát thủ công có thể gây thiếu hụt tài nguyên nếu quy mô mô hình tăng lên trong tương lai.
- **Chi phí khởi tạo hạ tầng còn lại:** Sau khi giảm nút thắt I/O đĩa, cold-start Optimized vẫn còn khoảng 17.35s (P99 trung bình). Ước lượng phần còn lại sau thời gian nạp model (1.38s) là khoảng 16s, chủ yếu thuộc quy trình lập lịch container, khởi tạo runtime Python và *deserialize* — nằm ngoài phạm vi tối ưu của cơ chế Weight-Sharing hiện tại.
- **Quy mô mẫu thống kê:** Thử nghiệm cold-start đối chứng được thực hiện với $n=3$; với mẫu nhỏ, giá trị P99 gần với giá trị cực đại và mang tính minh họa ổn định tương đối hơn là suy luận thống kê chặt chẽ.

5.3 Hướng phát triển

Để nâng cao hơn nữa hiệu năng và tính thực tiễn của hệ thống, các hướng phát triển trong tương lai bao gồm:

1. **Lưu trữ/cache phân tán:** Nghiên cứu tích hợp các giải pháp như Alluxio để chia sẻ trọng số trên nhiều node Kubernetes trong môi trường thực tế.
2. **Tối ưu hóa dung lượng mô hình:** Áp dụng các kỹ thuật nén mô hình như Quantization (INT8, FP16) hoặc Pruning nhằm giảm kích thước tệp trọng số, từ đó rút ngắn thời gian *deserialize* và tiết kiệm RAM.
3. **Cơ chế Pre-warming thông minh:** Kết hợp scale-to-zero với bộ dự báo lưu lượng để chủ động khởi tạo Pod trước khi lưu lượng bùng nổ, nhằm giảm hoặc tránh hoàn toàn cold-start cho người dùng cuối.
4. **Tối ưu hóa Runtime:** Đánh giá chuyển đổi sang các engine suy diễn hiệu năng cao hơn như NVIDIA Triton Inference Server hoặc ONNX Runtime để giảm thiểu chi phí của môi trường Python/FastAPI.

Tài liệu tham khảo

- [1] Knative Project Authors, “Knative Serving Overview,” *Knative Documentation*, 2026. [Online]. Available: <https://knative.dev/docs/serving/>
- [2] The Kubernetes Authors, “Kubernetes Documentation Reference,” *Kubernetes Official Docs*, 2026. [Online]. Available: <https://kubernetes.io/docs/>
- [3] Ultralytics LLC, “YOLO-World Open-Vocabulary Object Detection Docs,” *Ultralytics Documentation*, 2026. [Online]. Available: <https://docs.ultralytics.com/>
- [4] Grafana Labs, “k6 Load Testing Core Documentation,” *Grafana k6 Docs*, 2026. [Online]. Available: <https://grafana.com/docs/k6/>